

Telegram Automation Bridge

Документация проекта и исходный код системы автоматизации (Userbot)

1. Описание проекта

Telegram Automation Bridge — это специализированное программное решение на базе библиотеки **TeLethon** (клиентское API Telegram), предназначенное для автоматической омниканальной рассылки информационных материалов, коммерческих предложений и прайс-листов клиентам. Интеграция связывает мессенджер Telegram и традиционную электронную почту (SMTP) через единый интерфейс управления.

В отличие от классических ботов, создаваемых через BotFather, данная система функционирует в режиме **Userbot** (от имени реального профиля пользователя). Главное бизнес-преимущество такого подхода — возможность **инициировать диалог с клиентом первыми** по его номеру телефона без необходимости предварительного старта бота со стороны пользователя.

2. Архитектура и логика работы

Система разворачивается на выделенном сервере (VPS) и работает непрерывно в режиме ожидания событий (сообщений). Логика обработки делится на два основных сценария внутри одной приватной управляющей группы:

- **Обновление целевого файла (Медиа):** При отправке любого документа (Excel, PDF, Word) в управляющий чат, бот автоматически очищает локальное хранилище на сервере, скачивает этот файл и сохраняет его. Все последующие отправки клиентам будут содержать именно этот актуальный документ.
- **Рассылка контактов (Текст):** При отправке текстового сообщения скрипт с помощью регулярных выражений мгновенно сканирует текст на наличие номеров телефонов и адресов электронной почты.

Гибкость отправки:

- Если в сообщении распознан только номер телефона — документ отправляется исключительно в Telegram.
- If только Email — отправка идет только на почту.
- При наличии обоих контактов — отправка дублируется по обоим каналам связи одновременно.

3. Технические требования и зависимости

Для стабильной работы системы в режиме 24/7 требуется виртуальный сервер (VPS) под управлением операционной системы семейства Linux (рекомендуется Ubuntu 22.04 LTS или новее) с установленным интерпретатором Python версии 3.10+.

Библиотека	Назначение в архитектуре проекта
<code>telethon</code>	Взаимодействие с серверами Telegram по низкоуровневому протоколу MTProto.
<code>PySocks</code>	Обеспечение поддержки сетевого туннелирования для прокси-соединений.
<code>python-socks[asyncio]</code>	Асинхронная обработка сетевых сокетов через HTTP/SOCKS прокси для исключения блокировок основного потока.

4. Развертывание и запуск фоновой службы

Для того чтобы скрипт работал автономно и не закрывался при отключении от терминала, его необходимо настроить как системного демона `systemd`.

Шаг 1. Создание файла конфигурации службы

Выполните команду в терминале сервера:

```
sudo nano /etc/systemd/system/automation_bot.service
```

Шаг 2. Конфигурация службы

Вставьте в файл следующее содержимое (при необходимости скорректируйте пути к виртуальному окружению):

```
[Unit]
Description=Telegram Automation Bridge Daemon
After=network.target

[Service]
User=root
WorkingDirectory=/root/
ExecStart=/root/venv/bin/python /root/bot.py
Restart=always
RestartSec=5
```

```
[Install]
WantedBy=multi-user.target
```

Шаг 3. Активация управления процессом

Перезапустите менеджер конфигураций, включите автозапуск при старте сервера и запустите бота:

```
systemctl daemon-reload
systemctl enable automation_bot.service
systemctl start automation_bot.service
```

Полезные команды для мониторинга:

- Проверка текущего статуса службы: `systemctl status automation_bot.service`
- Просмотр логов работы в реальном времени: `journalctl -u automation_bot.service -f`

5. Безопасный исходный код системы (bot.py)

Ниже представлен полностью очищенный, готовый к публикации на GitHub исходный код программы. Перед запуском на сервере обязательно замените демонстрационные текстовые заглушки в блоке конфигурации на ваши реальные технические параметры.

```
# -*- coding: utf-8 -*-
import os
import re
import shutil
import smtplib

from email.message import EmailMessage
from telethon import TelegramClient, events

# =====
# БЛОК КОНФИГУРАЦИИ (ЗАПОЛНИТЕ ВАШИМИ РЕАЛЬНЫМИ ДАННЫМИ)
# =====
API_ID = 1234567 # Вставьте ваш числовой API ID с сайта my.telegram.org
API_HASH = 'ВАШ_API_HASH' # Вставьте ваш строковый API Hash в кавычках
PHONE_NUMBER = '+79990000000' # Номер телефона авторизованного аккаунта

# Параметры прокси-сервера (используются для стабильного обхода ограничений)
PROXY_SETTINGS = {
    "proxy_type": "http", # Допустимые форматы: "http", "socks5"
    "addr": "ИСПРАВЬТЕ_НА_IP_АДРЕС_ПРОКСИ",
    "port": 8080, # Укажите порт вашего прокси-сервера
    "username": "ЛОГИН_ПРОКСИ",
    "password": "ПАРОЛЬ_ПРОКСИ"
}

# Числовой идентификатор вашей приватной группы управления
COMMAND_CHAT_ID = -1000000000000

# Параметры авторизации почтового сервера SMTP
EMAIL_SENDER = 'your_email@example.com'
```

```

EMAIL_PASSWORD = 'ВАШ_КОНФИДЕНЦИАЛЬНЫЙ_ПАРОЛЬ_ОТ_ПОЧТЫ'

# Стандартный текст сопроводительного письма / сообщения
TEXT_CAPTION = (
    "Здравствуйте!\n"
    "Отправляю актуальные информационные материалы по вашему запросу.\n\n"
    "Буду рад сотрудничеству!"
)

# Папка для динамического сохранения файлов вложений
ATTACHMENTS_DIR = 'current_attachment'
if not os.path.exists(ATTACHMENTS_DIR):
    os.makedirs(ATTACHMENTS_DIR)

# =====
# ЛОГИКА РАБОТЫ СКРИПТА
# =====
client = TelegramClient('automation_production_session', API_ID, API_HASH,
    proxy=PROXY_SETTINGS)

def get_active_file():
    """Ищет и возвращает путь к сохраненному файлу рассылки"""
    files = os.listdir(ATTACHMENTS_DIR)
    if files:
        return os.path.join(ATTACHMENTS_DIR, files[0])
    return None

def send_email_sync(recipient_email, file_path):
    """Выполняет синхронную отправку Email сообщения через защищенный SMTP сеанс"""
    msg = EmailMessage()
    msg['Subject'] = 'Информационные материалы по вашему запросу'
    msg['From'] = EMAIL_SENDER
    msg['To'] = recipient_email
    msg.set_content(TEXT_CAPTION)

    if file_path and os.path.exists(file_path):
        with open(file_path, 'rb') as f:
            file_data = f.read()
            file_name_attach = os.path.basename(f.name)
            msg.add_attachment(file_data, maintype='application', subtype='octet-stream',
                filename=file_name_attach)
    else:
        return False, "Файл вложения не обнаружен на накопителе сервера"

    try:
        # Для использования TLS/порт 587 измените на smtpplib.SMTP с server.starttls()
        with smtpplib.SMTP_SSL('smtp.example.com', 465) as server:
            server.login(EMAIL_SENDER, EMAIL_PASSWORD)
            server.send_message(msg)
        return True, "Успешно"
    except Exception as e:
        return False, str(e)

@client.on(events.NewMessage(chats=COMMAND_CHAT_ID))
async def main_handler(event):
    text = event.raw_text or ""

    # 1. ОБРАБОТКА ЗАГРУЗКИ НОВОГО ФАЙЛА
    if event.message.media:
        status_msg = await event.reply("⌚ Скачиваю новый файл для рассылки...")

```

```

for filename in os.listdir(ATTACHMENTS_DIR):
    file_path = os.path.join(ATTACHMENTS_DIR, filename)
    if os.path.isfile(file_path):
        os.remove(file_path)

saved_path = await client.download_media(event.message.media, file=ATTACHMENTS_DIR +
'/'')
file_name = os.path.basename(saved_path)

await client.edit_message(event.chat_id, status_msg.id, f"✅ Новый файл успешно
сохранен:\n {file_name}\n\nТеперь он будет отправляться клиентам при вводе контакта.")

if not text:
    return

# 2. ПОИСК НОМЕРА И ПОЧТЫ С ПОМОЩЬЮ REGEX
phone_match = re.search(r'\+?\d[\d\-\s()]{9,}\d', text)
email_match = re.search(r'[a-zA-Z0-9_.\-]+\@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.\-]+\.', text)

if not phone_match and not email_match:
    if not event.message.media:
        await event.reply("❌ В сообщении не обнаружены ни номер телефона, ни email.")
    return

# 3. ПОДГОТОВКА ФАЙЛА К РАССЫЛКЕ
active_file = get_active_file()
if not active_file:
    await event.reply("❌ Ошибка: В системе нет файла для отправки!\nПожалуйста, отправьте
в эту группу файл, чтобы бот его запомнил.")
    return

log_status = f"⌚ Начинаю обработку заявки...\n📁 Файл: `{os.path.basename(active_file)}`
\n"
report_message = await event.reply(log_status)

# 4. ИНИЦИАЦИЯ ДИАЛОГА В TELEGRAM
if phone_match:
    raw_phone = phone_match.group(0)
    clean_phone = re.sub(r'\D', '', raw_phone)

    # Нормализация российских/международных мобильных форматов
    if clean_phone.startswith('8') and len(clean_phone) == 11:
        clean_phone = '+7' + clean_phone[1:]
    elif clean_phone.startswith('7') and len(clean_phone) == 11:
        clean_phone = '+' + clean_phone
    elif len(clean_phone) == 10:
        clean_phone = '+7' + clean_phone
    else:
        clean_phone = '+' + clean_phone

    log_status += f"\n📠 Telegram: Отправка на `{clean_phone}`...\n"
    try:
        await client.send_file(clean_phone, active_file, caption=TEXT_CAPTION)
        log_status += "✅ Telegram: Успешно отправлено!\n"
    except Exception as e:
        log_status += f"❌ Telegram: Ошибка ({str(e)})\n"
    else:
        log_status += "\n📌 Telegram: Номер не указан, пропущено.\n"

    await client.edit_message(event.chat_id, report_message.id, log_status)

```

```

# 5. ОТПРАВКА НА ЭЛЕКТРОННУЮ ПОЧТУ
if email_match:
    email_addr = email_match.group(0)
    log_status += f"✉ Почта: Отправка письма на `{email_addr}`...\n"
    await client.edit_message(event.chat_id, report_message.id, log_status)

    success, error_details = send_email_sync(email_addr, active_file)
    if success:
        log_status += f"✅ Почта: Письмо успешно доставлено!\n"
    else:
        log_status += f"❌ Почта: Сбой ({error_details})\n"
else:
    log_status += f"ℹ Почта: Адрес не указан, пропущено.\n"

    await client.edit_message(event.chat_id, report_message.id, log_status)

print("Система запущена. Ожидание файлов и команд...")
client.start(phone=PHONE_NUMBER)
client.run_until_disconnected()

```